

Appendices of “Semantics with Applications” with Gofer implementations

Jan Bækgaard Pedersen
matt@daimi.aau.dk

Morten Rhiger
mrhiger@daimi.aau.dk

(Revised and condensed by Kjell Post — kjell.post@mdh.se)

Appendix B

Introduction to Gofer Implementations

In this appendix we give the basic definitions needed to implement the various semantics definitions in **Gofer**. Essentially, this amounts to an implementation of the material of Chapter 1. The material developed in the this appendix are placed in a file “appB.gs” so that we can use it in the following appendices.

B.1 Abstract syntax and parsing

For **Num** we choose the primitive type **Int** of **Gofer**. For **Var** we choose lists of characters and so define the type synonym:

```
type Var = [Char]
```

For each of the syntactic categories **Aexp**, **Bexp** and **Stm** we define an algebraic data type taking into account the various possibilities mentioned by the BNF syntax of Section 1.2:

```
data Aexp =  N Int | V Var | Add  Aexp Aexp |  
            Mult Aexp Aexp | Sub  Aexp Aexp  
  
data Bexp = TRUE | FALSE | Eq Aexp Aexp | Le Aexp Aexp |  
            Neg Bexp | And Bexp Bexp  
  
data Stm  = Ass Var Aexp | Skip | Comp Stm Stm |  
            If Bexp Stm Stm | While Bexp Stm | Repeat Stm Bexp  
            Block [(Var,Aexp)] [(Var,Stm)] Stm | Call Var
```

Example B.1 The factorial statement is represented by the following abstract data type (which may be entered directly in **Gofer**):

```
factorial = Comp (Ass "y" (N 1))  
               (While (Neg (Eq (V "x") (N 1)))
```

```
(Comp (Ass "y" (Mult (V "y") (V "x")))
      (Ass "x" (Sub (V "x") (N 1)))))
```

Note that this is a representation of the *abstract syntax* of the statement

```
y := 1; while ~(x = 1) do (y := y * x; x := x - 1)
```

□

Exercise B.2 Specify an element of **Stm** that represents the statement constructed in Exercise 1.2 for computing n to the power of m . □

B.2 Evaluation of expressions

We shall first be concerned with the representation of values and states. The natural numbers **Z** will be represented by the type **Int** meaning that the semantic function $\mathcal{N}$ becomes trivial. The truth values **T** will be represented by the type **Bool** of booleans. So we define the type synonyms:

```
type Z = Int
type T = Bool
```

The set **State** is defined as the set of functions from variables to natural numbers so we define:

```
type State = Var -> Z
```

Example B.3 The state **s_init** that maps all variables except **x** to **0** and that maps **x** to **3** can be defined by

```
s_init "x" = 3
s_init y   = 0
```

Note that we encapsulate the specific variable name **x** in quotes whereas **y** can be any variable. □

The functions $\mathcal{A}$ and $\mathcal{B}$ will be called **a_val** and **b_val** in the implementation and they are defined by directly translating Table 1.1 and 1.2 into **Gofer**:

```
a_val :: Aexp -> State -> Z
b_val :: Bexp -> State -> T

a_val (N n) s      = n
a_val (V x) s      = s x
a_val (Add a1 a2) s = (a_val a1 s) + (a_val a2 s)
a_val (Mult a1 a2) s = (a_val a1 s) * (a_val a2 s)
a_val (Sub a1 a2) s = (a_val a1 s) - (a_val a2 s)
```

```

b_val TRUE s      = True
b_val FALSE s     = False
b_val (Eq a1 a2) s = True,  if a_val a1 s == a_val a2 s
                  = False, if a_val a1 s /= a_val a2 s
b_val (Le a1 a2) s = True,  if a_val a1 s <= a_val a2 s
                  = False, if a_val a1 s > a_val a2 s
b_val (Neg b) s    = True,  if b_val b s == False
                  = False, if b_val b s == True
b_val (And b1 b2) s = True,  if b_val b1 s == True &&
                           b_val b2 s == True
                  = False, if b_val b1 s == False ||
                           b_val b2 s == False

```

Exercise B.4 Construct an algebraic data type for the binary numerals considered in Section 1.3. Define a function `n_val` that associates a number (in the decimal system) to each numeral. $\square$

Exercise B.5 Define functions

```

fv_aexp :: Aexp -> [Var]
fv_bexp :: Bexp -> [Var]

```

computing the set of free variables occurring in an expression. Ensure that each variable occurs at most once in the resulting list. $\square$

Exercise B.6 Define functions

```

subst_aexp :: Aexp -> Var -> Aexp -> Aexp
subst_bexp :: Bexp -> Var -> Aexp -> Bexp

```

implementing the substitution operation, that is `subst_aexp a y a0` constructs $a[y \mapsto a_0]$ and `subst_bexp b y a0` constructs $b[y \mapsto a_0]$. $\square$

Appendix C

Operational Semantics in Gofer

In this section we implement the natural semantics and the structural operational semantics of Chapter 2 in **Gofer** and show how similar techniques can be used to implement an interpreter for the abstract machine and the code generation of Chapter 3.

We shall need the definitions from last section. Since the syntax of **Gofer** does not contain an `include` command the file must either be included manually or a project-file must be written. We make a project file “`appC.gp`” containing

```
appB.gs
appC.gs
```

To use the material one must enter

```
:p appC.gp
```

The material developed in this appendix must be placed in the file “`appC.gs`”.

In Chapter 2 we distinguish between two kinds of configurations, intermediate configurations and final configurations. This is captured by the algebraic data type:

```
data Config = Inter Stm State | Final State
```

In the next section we shall show how the natural semantics can be implemented and after that we shall turn to the structural operational semantics.

C.1 Natural semantics

Corresponding to the relation $\rightarrow$ in Section 2.1 we shall introduce a function `ns_stm` of type

```
ns_stm :: Config -> Config
```

The argument of this function corresponds to the left-hand side of $\rightarrow$ whereas the result produced corresponds to the right-hand side of the relation. This is possible because Theorem 2.9 shows that the relation is deterministic. The definition of `ns_stm` follows closely the definition of $\rightarrow$ in Table 2.1:

```

update s x v y = v,    if x == y
                = s y, otherwise

ns_stm (Inter (Ass x a) s)
    = Final (update s x (a_val a s))

ns_stm (Inter (Skip) s)
    = Final s

ns_stm (Inter (Comp ss1 ss2) s)
    = Final s''
    where Final s'  = ns_stm (Inter ss1 s)
          Final s'' = ns_stm (Inter ss2 s')

ns_stm (Inter (If b ss1 ss2) s)
    = Final s', if b_val b s
    where Final s' = ns_stm (Inter ss1 s)

ns_stm (Inter (If b ss1 ss2) s)
    = Final s', if not(b_val b s)
    where Final s' = ns_stm (Inter ss2 s)

ns_stm (Inter (While b ss) s)
    = Final s'', if b_val b s
    where Final s'  = ns_stm (Inter ss s)
          Final s'' = ns_stm (Inter (While b ss) s')

ns_stm (Inter (While b ss) s)
    = Final s, if not(b_val b s)

```

Note that in the axiom for assignment, `update s x v` corresponds to $s[x \mapsto v]$. The semantic function $\mathcal{S}_{\text{ns}}$ can now be defined by

```

s_ns ss s = s'
    where Final s' = ns_stm (Inter ss s)

```

Example C.1 We can execute the factorial statement (see Example B.1) from the state `s_init` mapping `x` to **3** and all other variables to **0** (see Example B.3). The final state `s_fac` is obtained as follows:

```
s_fac = s_ns factorial s_init
```

To get the value of `y` we evaluate `s_fac "y"`. □

Exercise C.2 Extend the definition of `ns_stm` to include the `repeat`-construct. □

Exercise C.3 Define an algebraic data type `Deriv_tree` representing the derivation trees of the natural semantics. Construct a variant of the function `s_ns` of type


```
s_ns :: Stm -> State -> Deriv_tree
```

that constructs the *derivation tree* for a given statement and state rather than just the final state. Apply the function to some example statements. $\square$

C.2 Structural operational semantics

When specifying the structural operational semantics we shall need to test whether $\Rightarrow$ produces an intermediate configuration or a final configuration. So we shall introduce the function `is_final` defined by:

```
is_Final (Inter ss s) = False
is_Final (Final s)    = True
```

Corresponding to the relation $\Rightarrow$ we define the function `sos_stm` of type:

```
sos_stm :: Config -> Config
```

As in the previous section the argument of this function will correspond to the configuration on the left-hand side of the relation $\Rightarrow$ and the result will correspond to the right-hand side. Again this implementation technique is only possible because the semantics is deterministic (Exercise 2.22). The definition of `sos_stm` follows Table 2.2 closely:

```
sos_stm (Inter (Ass x a) s)
  = Final (update s x (a_val a s))

sos_stm (Inter (Skip) s) = Final s

sos_stm (Inter (Comp ss1 ss2) s)
  = Inter (Comp ss1' ss2) s', if not(is_Final (sos_stm (Inter ss1 s)))
    where Inter ss1' s' = sos_stm (Inter ss1 s)

sos_stm (Inter (Comp ss1 ss2) s)
  = Inter ss2 s', if is_Final(sos_stm (Inter ss1 s))
    where Final s' = sos_stm (Inter ss1 s)

sos_stm (Inter (If b ss1 ss2) s)
  = Inter ss1 s, if b_val b s

sos_stm (Inter (If b ss1 ss2) s)
  = Inter ss2 s, if not(b_val b s)

sos_stm (Inter (While b ss) s)
  = Inter (If b (Comp ss (While b ss)) Skip) s
```

The function `sos_stm` implements one step of the computation. The function `deriv_seq` defined below will determine the complete derivation sequence (*even if it is infinite!*).

```
deriv_seq (Inter ss s)
  = (Inter ss s) : (deriv_seq (sos_stm (Inter ss s)))
deriv_seq (Final s) = [Final s]
```

The semantic function $\mathcal{S}_{\text{SOS}}$ can now be defined by the **Gofer** function `s_sos`:

```
s_sos ss s = s'
  where Final s' = last (deriv_seq (Inter ss s))
```

Example C.4 The derivation sequence obtained by executing the factorial statement on the state `s_init` can now be obtained as follows:

```
fac_seq = deriv_seq (Inter factorial s_init)
```

We may want to inspect this in more detail and in particular we may be interested in the values of the variables `x` and `y` in the various intermediate states. To facilitate this we use the function

```
show_seq fv l = layn (map show_config l)
  where show_config (Final s) =
    "final state:\n" ++ layn (map (show_val s) fv)
    show_config (Inter ss s) =
      (show ss) ++ "\n" ++ layn (map (show_val s) fv)
    show_val s x = " s(" ++ x ++ ")=" ++ show (s x)
```

The function call `show_seq ["x",y"] fac_seq` will for each configuration in the derivation sequence `fac_seq` list the statement part and the values of `x` and `y` in the state part. The final state of the derivation sequence can be obtained from

```
s_fac' = s_sos factorial s_init
```

and the value obtained for `y` is obtained by executing `s_fac' "y"`. □

Exercise C.5 Extend the definition of `sos_stm` to include the `repeat`-construct. □

C.3 Extensions of While

The implementation of the natural semantics of **While** in Section C.1 will now be extended to the procedure language **Proc** of Section 2.5. Rather than presenting a fully worked out implementation we shall give detailed instructions for how to construct it. We shall pay attention to the semantics of **Proc** with static scope rules for variables as well as procedures.

Exercise C.6 The first step will be to define the data types needed to represent the semantics of **Proc**.

- Define algebraic datatypes `Dec_V` and `Dec_P`

```
type Dec_V = [(Var, Aexp)]
type Dec_P = [(Var, Stm)]
```

- Define the algebraic type `Loc` to be `Int` such that locations will be integers. Define the function

```
new :: Loc -> Loc
```

such that `new` increments its argument by one.

- define algebraic types `Env_V` and `Env_P` corresponding to **Env_v** and **Env_p**. Define the function

```
upd_P :: (Dec_P, Env_V, Env_P) -> Env_P
```

corresponding to `updp`.

- Finally, we need a type `Store` corresponding to **Store**. There are at least three possibilities: One possibility is to define

```
data Loc' = Loc loc | Next
type Store = Loc' -> Z
```

as this will correspond closely to the definition of **Store**. Alternatively, one may identify the special token 'next' with location `0` and then simply define

```
type Store = Loc -> Z
```

The third possibility is to define

```
type Store = (Loc -> Z, Loc)
```

where the second component corresponds to the value of 'next'. Choose a method that seems appropriate to you. □

Exercise C.7 Finally we turn towards the transition system. We begin by implementing the transition system for variable declarations:

- Define an abstract data type `Config_D` for the configurations of the transition system for variable declarations.
- Then define a function

```
ns_dec_V :: Config_D -> Config_D
```

corresponding to the relation $\rightarrow_D$.

Now we turn to the transition relation for statements:

- Define an abstract data-type `Config_P` corresponding to the configurations $\langle S, sto \rangle$ and sto of the transition system.
- Next define a function

```
ns_stm :: (Env_V, Env_P) -> Config_P -> Config_P
```

Corresponding to the relation $\rightarrow$.

Finally define a function

```
s_ns :: Stm -> Store -> Store
```

that calls `ns_stm` with appropriately initialized environments. Use the function on various example statements in order to ensure that the implementation works as intended. $\square$

Exercise C.8 Modify the implementation above to use dynamic scope rules for variable declarations as well as procedure declarations. $\square$

It is more problematic to extend the implementation to handle the constructs of Section 2.4:

Exercise C.9 Discuss how to extend the implementation of the natural semantics in Section C.1 to incorporate the construct considered in Section 2.4. $\square$

Exercise C.10 Discuss how to extend the implementation of the structural operational semantics of Section C.2 to incorporate the constructs considered in Section 2.4. $\square$

C.4 Provably correct implementation

Rather than presenting a fully worked out **Gofer** script we shall provide exercises showing how to develop an implementation corresponding to Chapter 3.

Exercise C.11 We need some data types to represent the configurations of the abstract machine:

- Define an algebraic data type `Am_ins` for representing instructions and define the type synonym

```
type Am_code = [Am_ins]
```

for representing code.

- Define an algebraic data type **Stack_values** representing the elements that may be on the evaluation stack and define the type synonym

```
type Stack = [Stack_values]
```

- Define a type **Storage** representing the storage.

Finally define

```
type Am_config = (Am_code, Stack, Storage)
```

for the configurations of **AM**. □

Exercise C.12 We can then turn to the semantics of the abstract machine instructions. For this we proceed in three stages:

- First define a function **am_step** of type

```
am_step :: Am_config -> Am_config
```

implementing Table 3.1.

- We shall also be interested in the computation sequence of **AM** so define a function

```
am_comp_seq :: Am_code -> Storage -> [Am_config]
```

that given a sequence of instructions and an initial storage will construct the corresponding computation sequence.

- Finally define a function **run** corresponding to the function $\mathcal{M}$ of Chapter 3.

This provides us with an interpreter for **AM**. What happens if we enter a stuck configuration? □

Exercise C.13 Finally, we implement the code generation functions:

- Define functions corresponding to $\mathcal{CA}$, $\mathcal{CB}$ and $\mathcal{CS}$.
- Define a function **am_stm** corresponding to the function $\mathcal{S}_{\text{am}}$.

Apply the construction to a couple of examples to verify that everything works as expected. □

Exercise C.14 Modify the implementation to use the abstract machine **AM₂** of Exercise 3.8 and 3.17 rather than **AM**. □

Appendix D

Denotational semantics in Gofer

In this section we implement the denotational semantics of Chapter 4 in **Gofer** and show how similar techniques can be used to implement the static program analysis of Chapter 5. We shall need the definitions from Appendix B so we begin by writing a project file “appD.gp” containing

```
appB.gs
appD.gs
```

The material developed in this appendix must then be placed in the file “appD.gs”.

D.1 Direct style semantics

In the implementation we shall rely on the built-in function ‘.’ which denotes function composition.

The theoretical foundation of **Gofer** is closely related to the theory developed in Chapter 4 (although it is outside the scope of this book to go further into this). One of the consequences of this is that the fixed point operation can be implemented in a very simple way:

```
fix ff = ff (fix ff)
```

The function $\mathcal{S}_{ds}$ can now be implemented by the function

```
s_ds :: Stm -> State -> State
```

A straightforward rewriting of Table 4.1 gives:

```
s_ds (Ass x a)      = \s -> update s x (a_val a s)
                      where update s x v y = v,   if x == y
                      = s y, otherwise
```

```
s_ds Skip           = \s -> s
```

```
s_ds (Comp ss1 ss2) = (s_ds ss2) . (s_ds ss1)
```

```

s_ds (If b ss1 ss2) = \s -> if b_val b s then s_ds ss1 s else s_ds ss2 s

s_ds (While b ss)    = fix (\g s -> if b_val b s then g (s_ds ss s) else s)

```

Example D.1 Returning to the factorial statement we can apply its denotation to the initial state `s_init` as follows:

```

s_final = s_ds factorial s_init

```

Exercise D.2 We may be interested in the various iterands of the fixed point. Rewrite the semantics equations above so that each fixed point is unfolded at most n times where n is an additional parameter to the functions. Give examples showing that if the value of n is sufficiently large then we get the same result as above. □

Exercise D.3 Extend the definition above to handle the `repeat`-construct. □

D.2 Extensions of While

It is fairly straightforward to extend the implementation to handle the procedure language and the exception language of Section 4.5.

Exercise D.4 Modify the above implementation to use environments and stores and extend it to implement the semantics of the language **Proc** of Section 4.5. □

Exercise D.5 Modify the above implementation to use continuations and extend it to handle the language **Exc** of Section 4.5. □

D.3 Static program analysis

Rather than presenting a fully worked out **Gofer** script performing the dependency analysis we shall provide a rather detailed list of instructions for how to develop such an implementation.

Exercise D.6 The first step will be to implement the complete lattices **P** and **PState** and the operations on them:

- Define an algebraic data type **Property** representing the set **P** of properties and define a function `p_lub` corresponding to $\sqcup_P$.
- Define a type synonym **Pstate** representing the property states. Define the special property states `INIT` and `LOST`. Define a function `pstate_lub` corresponding to $\sqcup_{Ps}$. □

Exercise D.7 We can turn to the semantic equations defining the analysis:

- Define the functions

`p_aexp :: Aexp -> Pstate -> Property`

corresponding to $\mathcal{PA}$ and

`p_bexp :: Bexp -> Pstate -> Property`

corresponding to $\mathcal{PB}$.

- Define the auxiliary function `cond_P` corresponding to cond_P .
- Define the function

`p_stm :: Stm -> Pstate -> Pstate`

corresponding to $\mathcal{PS}$ of Table 5.3. (You may use the result of Section 5.4 for this).

□

Exercise D.8 Implement the algorithm of Section 5.2 and apply the implementation to a couple of examples to verify that everything works as expected. □

Appendix E

From Miranda to Gofer by hand

In this appendix we give an informal description of the translation from **Miranda**-programs to **Gofer**-programs. In particular we describe what was necessary to change the appendices. We do not catch all differences but mainly those that are relevant to the previous appendices. For deeper explanations see [2] (Appendix D), [1] and [3].

E.1 Scripts and projects

In **Gofer** it is not possible to include a file from another file. If two or more files are mutually dependent a *project*-file must be made to be able to access them. Loading a **Miranda**-script may cause a number of files “`file1.m`”, “`file2.m`”, ..., “`filen.m`” to be read. These files are those found in the dependency graph. When translating the files to **Gofer** one must make a project file “`file.gp`” containing

```
file1.gs  
file2.gs  
...  
filen.gs
```

where each “`filei.gs`” is a **Gofer**-script corresponding to the **Miranda**-script “`filei.m`”.

E.2 Types and data types

In **Miranda** type synonym names as well as data type names must begin with a lower-case letter. Declarations of new type synonyms starts with the name of the type synonym, followed by the formal arguments (if any), a double-equal sign “`==`” and the type expression. Declaration of data types starts with the name of the data type, followed by the formal arguments (if any), followed by “`::=`” and the constructors which are separated by “`|`”. Constructor names must be capitalized.

In **Gofer** all type synonym names and data type names must start with a capital letter. This includes the basic types. Declaration of new type synonyms must start with the keyword **type**, the name of the new synonym, the possible formal arguments, an equal sign “`=`” and the type expression. Declaration of new abstract data types starts with

the keyword **data** followed by the name of the data type, the possible formal arguments, an equal sign “=” and the constructors separated by “|”. Constructor names must be capitalized too.

The syntax for type expressions are almost the same in the two languages. The differences has not influenced the translation of the appendices. We have therefore the correspondence

Miranda	Gofer
num	Int
bool	Bool
char	Char
string == [char]	type String = [Char]
sex ::= Male Female	data Sex = Male Female
set a == [a]	type Set a = [a]

E.3 Functions and expressions

Definition of functions have almost the same syntax in the two languages. They both allow mutually recursive functions in a syntax close to “mathematical notation” with pattern matching of actual function arguments. One difference is that **Gofer** requires the number of patterns (on the left-hand side) to be the same for all defining clauses for each particular function. Thus

```
binary      :: Int -> Int -> Int
binary 0    = id
binary n x  = n * x
```

is not a legal function declaration. There are however no problems if one rewrites the above to the equivalent

```
binary      :: Int -> Int -> Int
binary 0    = id
binary n    = \x -> n * x
```

This restriction has only influenced the development of the function **s_ds** in Appendix D. Both languages allow the **where** clause as a **let**-construct and they both allow guarded **if**-expression. **Gofer** accepts two kinds of syntax for the guarded **if** expression. In this paper we have chosen the one similar to **Miranda**.

Expressions have the same syntax in the two languages. They only differ on the set of predefined functions. This has only influenced the development of the function **show_seq** in Section C.2 where we use the **Gofer**-function **layn** as a substitute for the **Miranda**-function **lay**.

Bibliography

- [1] Ian Holyer. *Functional programming with Miranda*. UCL Press, 1993.
- [2] Mark Jones. *An Introduction to Gofer*, February 1995.
- [3] Jan Bækgaard Pedersen and Morten Rhiger. An introduction to the gofer functional programming environment. Report, DAIMI, Computer Science Department, Aarhus University, Ny Munkegade, 8000 Aarhus C, Denmark, 1995.
- [4] Torben Mogensen. Ratatosk. a parser generator and scanner generator for gofer. Anonymos ftp: `ftp.diku.dk` file `pub/diku/dists/Ratatosk.tar.Z`, 1993.
- [5] Hanne Riis Nielson and Flemming Nielson. *Semantics with Applications, a formal introduction*. Wiley Professional Computing. John Wiley and Sons, 1992.